

Appunti del corso di **Basi di Di Dati**

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis

Matteo Drago

Indice

1	Introduzione	1
1.1	Perchè studiare i PL?	1
1.2	Cosa sono i PL?	2
1.3	Cosa significa programmare?	2
1.4	Contesti di sviluppo	3
1.5	Classificazione dei PL	3
1.6	Implementazione di un PL	4
2	Comandi	5

Introduzione

Il problema principale dell'informatica consiste nel voler far eseguire ad una macchina algoritmi che manipolano dati.



Il primo meccanismo che viene associato alla storia dei computer è la **macchina di Babbage**, che può essere visto come il primo "computer" perchè nasce per eseguire calcoli matematici complessi in modo automatico.

Sucessivamente, c'è un'evoluzione e il computer viene riconosciuto come macchina programmabile (dare delle istruzioni alla macchina per ottenere un determinato risultato).

In questo contesto nasce **Enigma**, uno strumento per decodificare un codice mediante un algoritmo di forza bruta che tenta tutte le combinazioni.

La prima architettura che darà origine all'architettura dei computer moderni è l'**architettura di Von Neumann**, un'architettura hardware che comprende solo il linguaggio binario, un linguaggio costituito esclusivamente da stringhe di bit (0 e 1) e che può essere utilizzato per codificare qualunque altro linguaggio, ma non è comprensibile dall'essere umano.

Questo fa sì che l'uomo per poter interagire e programmare macchine, necessita di un'evoluzione dei linguaggi, ha bisogno di costruire un linguaggio comprensibile in modo più immediato.

1.1 Perchè studiare i PL?

Le motivazioni sono diverse:

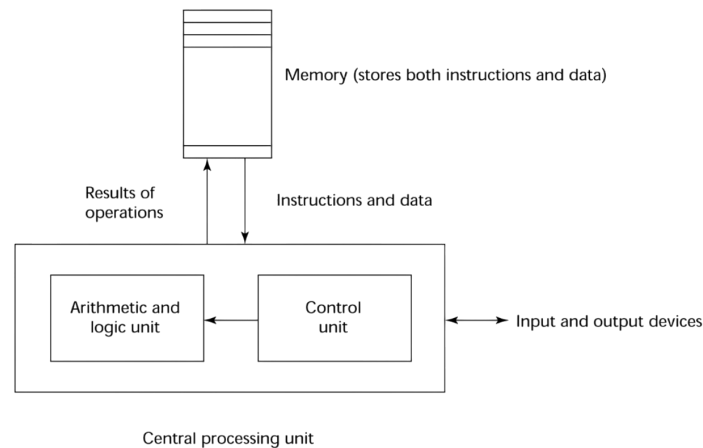
- ▷ migliorare la capacità di esprimere idee in forma algoritmica
- ▷ scegliere appropriatamente un linguaggio
- ▷ migliorare la capacità di studiare o progettare nuovi linguaggi o costrutti
- ▷ comprendere meglio il significato dell'implementazione dei linguaggi di programmazione

1.2 Cosa sono i PL?

La nascita e l'evoluzione dei linguaggi di programmazione è, anche, dovuto, all'architettura della macchina che bisogna programmare.

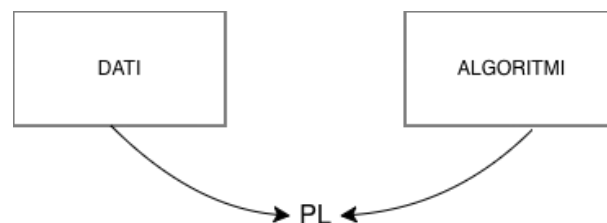
I computer moderni nascono dall'architettura di Von Neumann.

Questa è una tipologia di architettura hardware con programma memorizzato, dove i dati e istruzioni condividono la stessa area di memoria.



1.3 Cosa significa programmare?

Il linguaggio di programmazione deve essere uno strumento che permette di descrivere algoritmi e rappresentare i dati.



Il linguaggio binario è un linguaggio di programmazione in quanto permette di rappresentare algoritmi e dati, ma non è umanamente utilizzabile o interpretabile, in quanto potrebbe essere usata come dato o un dato interpretato come istruzione.

Ciò significa, che un linguaggio di programmazione deve permettere di superare queste difficoltà, fornendo una chiara distinzione tra dato e istruzione.

Definizione: algoritmo

Un **algoritmo** è una sequenza infinita di passi di computazione, ovvero scompone un calcolo complesso in passi elementari.

Un algoritmo è un concetto astratto che trova forma concreta in un programma (sequenza finita di istruzioni).

Definizione: dati

I **dati** sono informazioni memorizzate concretamente in celle di memoria e astratte in elementi che il linguaggio può manipolare (variabili).

Definizione: programma

Un **programma** è la concretizzazione di un algoritmo che manipola dati.

Definizione: linguaggio di programmazione

Un **linguaggio di programmazione** è una notazione per scrivere un programma, ovvero serve a dare forma concreta agli algoritmi.

Il programma, quindi, costituisce la **sintassi**.

Un programma è necessariamente un oggetto finito (sempre concreto), mentre il calcolo di una funzione richiede l'esecuzione di un insieme di passi primitivi, che, però possono essere infiniti.

Ciò significa che un programma è una rappresentazione finita di un insieme, potenzialmente infinito, di passi primitivi di computazione.

Nota

I linguaggi di programmazione devono dare specifica finita di oggetti potenzialmente infiniti.

1.4 Contesti di sviluppo

Il contesto in cui il linguaggio deve operare determina le funzionalità che deve gestire efficacemente:

- ▷ **applicazioni scientifiche** (es. Fortran)
- ▷ **applicazioni economiche** (es. COBOL)
- ▷ **applicazioni artificiali** (es. LISP)
- ▷ **programmazione di sistemi** (es. C)
- ▷ **web software** (es. HTML (markup), PHP (scripting), ecc.)

1.5 Classificazione dei PL

I linguaggi di programmazione si possono classificare in:

- ◇ **basso livello**: linguaggi con caratteristiche specifiche dipendenti dall'architettura che si sta programmando (es. linguaggio binario, Assembly)
- ◇ **alto livello**: linguaggi che permettono una programmazione strutturata in cui dati e istruzioni hanno rappresentazioni diverse

I linguaggi ad alto livello si possono classificare in base a:

- **paradigma di programmazione**
 - linguaggi procedurali: codice diviso in subrutine che eseguono compiti semplici basati su salti condizionali (goto) [es. COBOL]
 - linguaggi strutturati (imperativi): procedurale senza goto (es. C, Pascal, ADA, ec..)
 - linguaggi orientati agli oggetti: classe elemento centrale della programmazione. Un linguaggio deve avere tre proprietà:
 - * incapsulamento
 - * ereditarietà
 - * polimorfismo
 - linguaggi dichiarativi: basati su componenti della matematica
 - * logici: basati sulla logica proposizionale del primo ordine, in cui si effettuano calcoli tramite dimostrazione di fatti (es. PROLOG)
 - * matematici: basati sul concetto di applicazione di funzione (es. LISP, ML)
- **termini generazionali**
 - I° generazione:

↓
 - II° generazione:

1.6 Implementazione di un PL

Implementare un linguaggio significa eseguire un programma nel linguaggio di programmazione in una macchina.

Sicuramente, il modo in cui deve essere implementato è direttamente collegato all'architettura che deve programmare, ma anche al paradigma (metodologie) che il linguaggio di programmazione utilizza per costruire programmi.

Implementare un linguaggio significa realizzare la macchina (astratta) che interpreta il linguaggio.

Nota

La macchina è astratta perchè, per rappresentare la macchina fisica (concreta), utilizza un linguaggio che è più astratto del linguaggio binario.

Definizione: macchina astratta

Dato un linguaggio L di programmazione, la **macchina astratta** M_L per L , è un insieme di strutture dati e algoritmi (è un programma) che permettono di memorizzare ed eseguire i programmi scritti in L .

Comandi

I **comandi** sono la categoria sintattica che permettono di rappresentare trasformazioni irreversibili dello stato.

L'astrazione dello stato della macchina è rappresentata dalla memoria.

Definizione: memoria

La **memoria** è l'insieme delle associazioni tra locazioni e valori (es. array illimitato: indice = locazione, valore = contenuto della locazione).

La trasformazione dello stato, avviene tramite un meccanismo definito **assegnamento**.

Definizione: assegnamento

L'**assegnamento** è un comando che permette di modificare il contenuto della memoria.

Ne deriva che non è pensabile che si possa utilizzare direttamente la locazione di memoria. Le locazioni vengono riferite attraverso identificatori chiamate variabili.

Definizione: variabili

Le **variabili** sono identificatori associati a locazioni, che a loro volta, nella memoria, sono associate a valori che possono cambiare durante l'esecuzione.

Per il momento, quindi, supponiamo che le variabili siano nomi astratti per le celle di e supponiamo di associare direttamente il valore (modificabile) a una variabile.

Vediamo alcuni aspetti dell'uso delle variabili che rendono questo modello poco adatto:

- ▷ quando ci riferiamo a un valore modificabile mediante una variabile, lo possiamo fare sia per accedere al suo valore in lettura, sia per modificare il suo valore
- ▷ per riferirsi a oggetti da modificare potremmo utilizzare funzioni della variabile e non il semplice nome (es. array $a[x+1]$)
- ▷ gli identificatori possono essere ridefiniti causando modifiche reversibili all'ambiente. Ciò significa che per un certo intervallo di tempo, una variabile potrebbe avere in altro significato, mentre il valore che aveva prima della ridefinizione non deve andare perso perchè potrebbe ritornare valido (es. uscita da una procedura)

Introduciamo, quindi, la locazione come astrazione della memoria fisica e la variabile come astrazione della locazione.

Significa che introduciamo qualcosa di intermedio tra il nome della variabile e il nome riferito, che modella la memoria nella sua visione ideale (illimitata).

In questo modo è possibile che una variabile possa avere indirizzi differenti in momenti diversi dall'esecuzione, oppure che una variabile possa avere differenti indentificatori in punti diversi del programma.

Si distinguono l'associazione tra nome e locazione che non cambia in modo reversibile durante l'esecuzione e l'associazione tra locazione e valore che cambia in modo irreversibile durante l'esecuzione.

Definizione: locazione

La **locazione** è il contenitore di valori che possono essere inizializzati, indefiniti o definiti.

Definizione: memoria

La **memoria** è la collocazione delle associazioni tra locazioni e valori contenuti, che possono essere costantemente cambiati.

Questo significa che la locazione viene creata da una dichiarazione.

Inoltre, la locazione ha un tempo di vita determinato dallo scope della dichiarazione.

Definizione: identificatore costante

un **identificatore costante** è un identificatore che può assumere un valore predefinito e non è ulteriormente modificabile.

Definizione: identificatore variabile

Un **identificatore variabile** è un identificatore il cui valore può essere aggiornato.